



February 20, 2025, Brno
Author: David Procházka

Computer Graphics

Graphic Application
Development

- MENDELU
- Faculty
- of Business
- and Economics

Table of content

- 1 Course organization
- 2 GPU Architecture
- 3 OpenGL
- 4 Summary

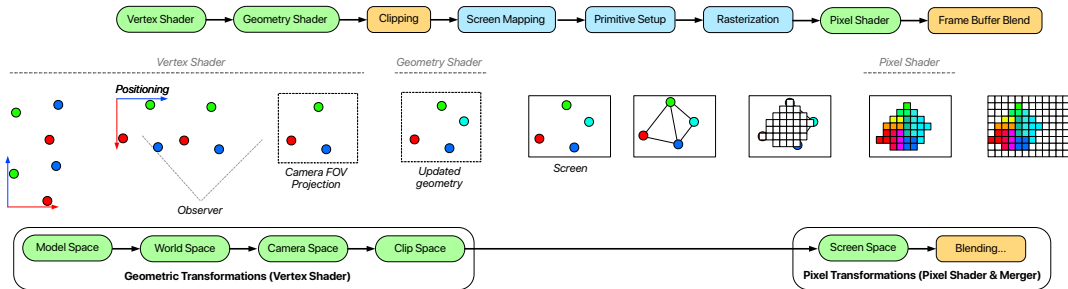
Course organization

- Our course has three key parts according to the use-case:
 - ① **OpenGL** a.k.a. low-level development (aerospace, embedded systems...),
 - ② **WebGL & libs** a.k.a. web development (games, Google Maps/Earth...),
 - ③ **Unreal Engine** a.k.a. contemporary multiplatform games development
- Requirements are in the syllabus (OpenGL app, project, exam)
- Beware of the obsolete literature!

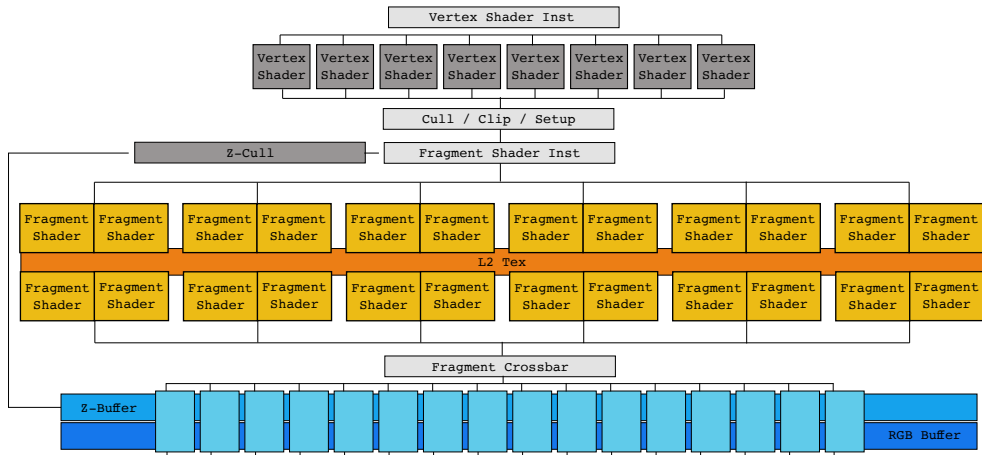
Table of content

- 1 Course organization
- 2 GPU Architecture**
- 3 OpenGL
- 4 Summary

Rendering pipeline

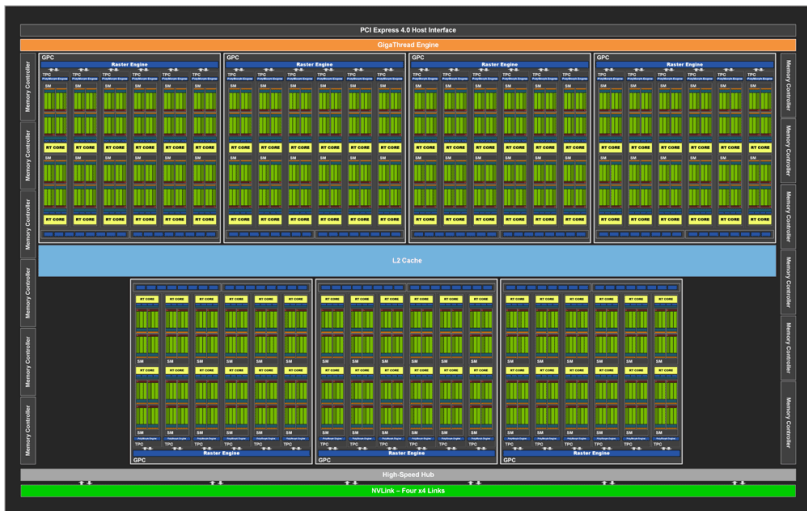


GPU in modern era (before nVidia Tesla)



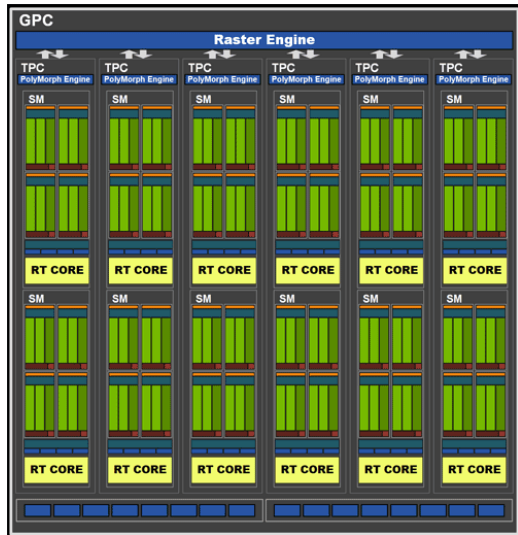
Source: <https://fabiansanglard.net/cuda/>

GPU Architecture: nVidia Ampere (GA102)



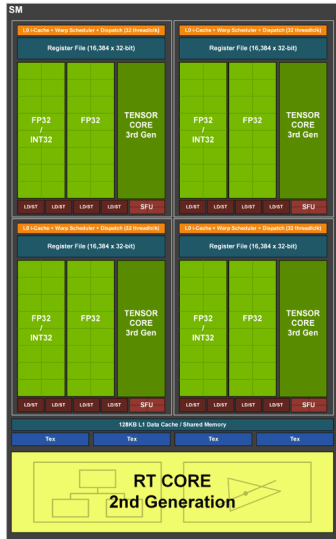
Graphics Processing Cluster (GPC)

- GA102 contains 7 GPCs (similar to *Shader Array*).
- GPC represents **complete rendering pipeline**.
- Structure: 1 raster engine, 6 Texture Processor Clusters (TPC) with 2 streaming multiprocessors (SM) for each TPC.

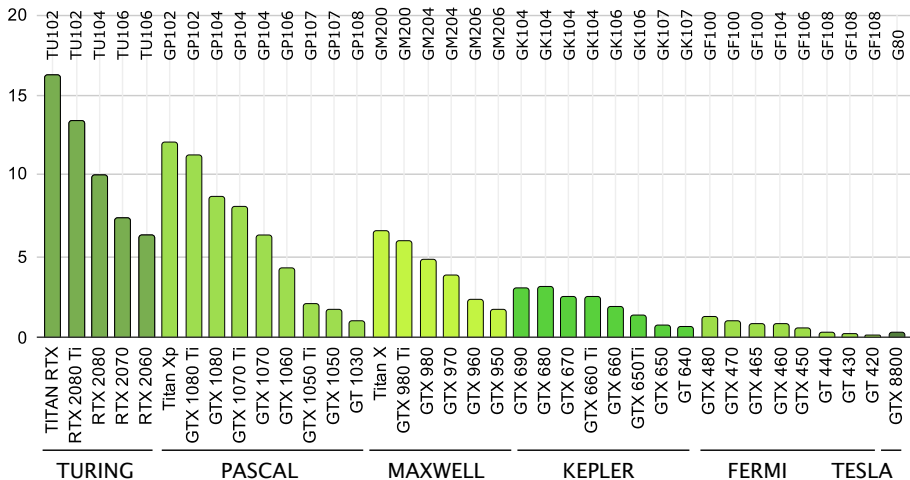


Streaming Multiprocessor (SM)

- Replaces previously used vertex/pixel shader units.
- 4 blocks of 32 CUDA Core units (FP32), 16 INT 32 units, Tensor Core unit (tensor, matrix, ML...),
- 4 Texture Mapping Units (TMU)
- RT Core (HW raytracing)



How fast is it? (Tflops/s per card)



Hardware raytracing: GTX vs RTX

- Common illumination method is **radiosity**. It's simple.
- **Raytracing** enables more precise illumination including sharp shadows, translucency, reflections etc.
- Can be computed in general cores, but dedicated RT cores are much faster.
- Currently supported just by nVidia cards.



API: communication with the GPU

- First standard: [OpenGL](#) by Khronos (1992)
- First competitor: [Direct X](#) by Microsoft (1995)
- Smartphone boom (2007): [OpenGL ES](#) (2003)
- Full-stack solution: [Metal](#) by Apple (2014)
- OpenGL successor: [Vulkan](#) by Khronos (2015)



Microsoft®
DirectX®



Vulkan

OpenGL®



Low-level vs hi-level approaches

- Low-level development: e.g. OpenGL+Qt, Metal+SwiftUI...
- Frameworks & libraries: e.g. SpriteKit+SwiftUI, WebGL+Three.js etc.
- Complex game engines: mostly Unreal Engine, Unity

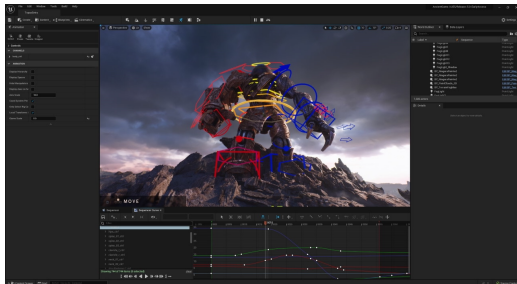


Table of content

- 1 Course organization
- 2 GPU Architecture
- 3 OpenGL**
- 4 Summary

Features

- API for 2D and 3D graphics (not complex SDK like DirectX).
- Platform and language independent (DirectX is for Windows and XBox, Vulkan for Windows and Linux, Metal for Apple platforms).
- Purely procedural (the code can be, however, object oriented).
- Many versions (OpenGL ES, OpenGL SC etc.)
- Currently suitable mostly for industry.
- From the 1992, maintained by Architecture Review Board (ARB), part of Khronos Group (<http://www.khronos.org>),
- Khronos Group: open, promoted members can vote etc.
- Promoted members: AMD, ARM, Apple, Epic Games, Ericsson, Freescale, Imagination, Intel, Nokia, NVIDIA, Qualcomm, Samsung, Sony, Texas Instruments.
- KG maintains many standards: OpenGL, OpenCL, OpenGL ES, OpenGL SC, COLLADA, WebGL, Vulkan, OpenML...

History

- **1980:** Iris GL – Silicon Graphics, SGI workstations,
- **1992:** OpenGL 1.0 – Mark Segal and Kurt Akeley,
- **2004:** OpenGL 2.0 – full-scale GPU support (shaders)
- **2008:** OpenGL 3.0 – anticipated OO model, [deprecation mechanism](#)
- **2010:** OpenGL 4.0 – tessellation on GPU, better shaders
- **2017:** OpenGL 4.6 – more efficient geometry processing and sh. execution etc.

Table of content

- 1 Course organization
- 2 GPU Architecture
- 3 OpenGL
- 4 Summary**

Takeaway

- What is the difference between API and library/engine?
- What APIs are currently used?
- What is rendering pipeline and how it works?
- What is a shared and what is it's role in rendering pipeline?